

Compliance Q&A RAG Project

A friendly guide to doing the exercise and running the real app 🥳

Welcome! This project walks you through building the core of a real question answering system, one that reads company policy documents and answers questions about them. This guide is not about writing code from scratch, it is about knowing exactly where things live and which buttons to press. Keep it next to you while you work.

Section 1: Two ways to do the exercise

You can complete this project through either path below. Pick whichever feels more comfortable, both lead to the same working app.

Path A

Notebook mode, the guided path

Open `notebook/rag_project_STUDENT.ipynb` (in Google Colab, or locally in Jupyter). Every exercise cell is marked with a 🧠 symbol and contains guided comments telling you exactly what to write, one small step at a time.

1. Fill in each 🧠 marked cell, following its comments.
2. Run the test cell right underneath it. Green checkmarks mean that piece works.
3. Keep going through Part 1 and Part 2 of the notebook.
4. At the end of each part, run the export cell. It writes two files for you: `solutions/part1_ingestion.py` and `solutions/part2_retrieval.py`.
5. Take those two files (if you worked in Colab, download them) and place them into the `solutions/` folder at the root of your copy of the repository, replacing the files already there.

Once both files are in place, the real app automatically uses your code. That is it, you are ready for Section 2.

Path B

Reference mode, the codebase path

Prefer to work straight in the code instead of a notebook? Open the `solutions/` folder in the repository. It contains two files:

- `solutions/part1_ingestion.py`, three functions to fill in
- `solutions/part2_retrieval.py`, four functions to fill in

Each function currently has a placeholder body. Read the real project code around it (files like `chunking.py`, `ingestion_core.py`, `loaders.py`, `retrieval_core.py`, `rag_core.py`) to understand what the function is supposed to do, then write your own version directly in the `solutions/` file.

The convenience of this path: you can implement the functions one at a time, in any order you like. The moment you finish a function and save the file, the app starts using *your* version of that one function immediately. Every function you have not gotten to yet keeps quietly working on the project's own correct reference code behind the scenes. So the app runs end to end at every stage, even when you are only halfway done.

No notebook needed for this path at all.

Section 2: Running the full app

Once your two files are ready (from either path above), here is how to see the whole thing running in a browser, step by step. Instructions are shown for both macOS and Windows wherever they differ.

1 Clone the repository

This command is the same on macOS and Windows. The repository is **public**, so no GitHub access needs to be granted, anyone can clone it. The command pins the project branch, which is where the course files live.

```
git clone --branch project --single-branch https://github.com/eth-fdd-fs26/FDD-WE5-public.git
cd FDD-WE5-public
```

2 Install the prerequisites

You need Python 3.12 or newer, the uv package manager, and Node.js (any recent version) for the frontend.

MACOS

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

WINDOWS (POWERSHELL)

```
powershell -ExecutionPolicy ByPass -c
"irm https://astral.sh/uv/install.ps1 | iex"
```

For Node.js, download the installer from nodejs.org on either system, that part is identical, just click through the graphical installer.

Then, from inside the cloned FDD-WE5-public folder (same command on both systems), install the Python dependencies:

```
uv sync
```

3 Your OpenRouter API key

The app needs an OpenRouter key to actually answer questions (it pays for the embedding model and the LLM). Get a free key at openrouter.ai.

Main way to use it: once the app is open in your browser (Step 5 below), there is a key field right in the page. Paste your key there. That is all you need for the normal app flow, no environment variable required.

Only if you also want to run the command line demo (`main.py`) or the notebook's optional live demo cell, set it as an environment variable instead:

MACOS / LINUX

```
export OPENROUTER_API_KEY=sk-...
```

WINDOWS (POWERSHELL)

```
$env:OPENROUTER_API_KEY="sk-..."
```

4 Place your notebook-generated files

If you worked through Path A (the notebook), copy the two files it exported into the `solutions/` folder at the root of the repository, replacing what is there:

```
solutions/part1_ingestion.py
solutions/part2_retrieval.py
```

If you worked through Path B, your edits are already in the right place.

5

Start the two servers

The app is two small programs running side by side: a backend and a frontend. Open two terminal windows. These commands are identical on macOS and Windows.

TERMINAL 1, THE BACKEND

```
uv run uvicorn server:app --reload
```

Runs at <http://localhost:8000>

TERMINAL 2, THE FRONTEND

```
cd frontend  
npm install  
npm run dev
```

Runs at <http://localhost:5173>

Open <http://localhost:5173> in your browser. Paste your OpenRouter key into the key field, upload documents or use the ones already loaded, and start asking questions. 🦉

Section 3: How the app works, in plain terms

You do not need to understand this to finish the exercise, but it helps to see the shape of the whole system. One class, `RAGCore`, sits in the middle and coordinates everyone else. It never does the detailed work itself, it just calls the right specialist for each job and passes the result along.

Class	What it actually does
RAGCore	The orchestrator. Built once when the app starts, it holds one of each class below and is the only thing the rest of the app talks to. "Ingest these documents", "answer this question", it decides who handles each request.
IngestionCore	The write side. Given a document, it splits it into smaller passages, turns each one into a vector, and hands them off to be stored.
RetrievalCore	The read side. Given a question, it searches for matching passages two different ways (by keyword and by meaning) and combines the results into one ranked list.
DBManager	Wraps Qdrant, the vector database. This is where every document passage and its vector actually lives, and where <code>RetrievalCore</code> reads them back from.
TextEmbedder	Turns text into a vector using an embedding model on OpenRouter. Used both when storing documents and when understanding a new question.
LLMClient	The last step. Sends the question together with the best matching passages to a language model on OpenRouter, and returns the written answer.

When you ask a question in the app: `RAGCore` asks `RetrievalCore` to find matching passages (`RetrievalCore` asks `TextEmbedder` to understand the question, and `DBManager` to fetch candidates), then `RAGCore` hands the best passages to `LLMClient`, which returns the final answer you see on screen.